`Devlog`: a classification package for NCq3 type cellular automata

~ Kristóf

Tuesday 20th October, 2026

1 Goal of the project

This project is about classifying all local spin-1 integrable models that admit the Hamiltonian density

$$h_{j,j+1,j+2} = \sum_{ab=0,1,2} (|a\rangle\langle a|)_j h_{j+1}^{f(a,b)} (|b\rangle\langle b|)_{j+2}, \tag{1}$$

where $h_j^{f(a,b)}$ denote some Hermitian matrices. The function $f(a,b) \in \{0,1,2\}$ is a model specific update rule, that determines whether h_{j+1} is actually acting on the jth site, or not.

The bird's-eye view workflow is the following:

- A general ansatz is made for the Hamiltonian density. Hermiticity is enforced by construction, via employing specific generating operators that span the whole Herm(3) space. As coefficients, only real numbers are allowed.
- We take it as established that the zero commutator of the first two dynamical charges of a system implies integrability (recent articles of Akihiro Hokkyo (2025), Mizuki Sanatani and Naoto Shiraishi (2026)). In the case of the particular ansatz:

$$[H, \mathcal{Q}_5] = 0. \tag{2}$$

- Enforcing the above conditon on an infinite/periodic chain, a number of third order polynomials are obtained, which then form the system of equations that need to be solved.
- After the system has been solved, we convert the solution strings back to Hamiltonian densities, then classify them based on a number of properties.

2 Rough structure of the package

The package should be divided to the following modules so that the components stay neatly ordered. Also, a certain hierarchy is imposed upon the modules: the `operators` module needs to be written early on, since equation generation, the ansatzes and a lot of other parts of the code depend on these basic building blocks.

1. **Operators**
   Contains every operator that will be used in the calculations, including: Pauli, spin-1 operators, projectors, etc.
2. **Algebra**
   Defines the exact operator algebra employing efficient sparse matrix computation techniques. Contains addition, multiplication, tensor products with identities, embedding an operator into a larger system, so on.

3. **Ansatz**
   Converts the inputted density-as-dictionary into an actual ansatz for the Hamiltonian while verifying all the required properties so that the model stays physical. At this point, certain symmetries can be imposed upon the model, restricting the parameter and solution space.

4. **Symmetry families**
   Contains the names and the restrictions for each symmetry family. The symmetries probably should not be implemented as adding more equations to an already large system.

5. **Equation generation**
   Takes the Hamiltonian density and $\tilde{h}$ as inputs, imposes the Reshetikhin condition on $H, \mathcal{Q}_5$, and outputs the set of equations that are obtained by forcing every component to be zero. Real and imaginary parts are treated as separate, real-valued equations: this way, the solution procedure can be carried out on a more restricted polynomial ring (e.g over $\mathbb{Q}$).

6. **Solver pipeline**
   A complex solving system that is built around providing the simplest, most physically restricted system of polynomials for a Gröbner basis solver. The algorithm runs on a CAS backend to optimize runtime.

7. **Back-conversion**
   Takes the solutions from the previous modules and converts them back into the Hamiltonian densities for which the ansatz has been made in the first place. It also prunes duplicate solutions, subclasses and operators that could be obtained by some transformation on an other element of the solution set.

2.1 *Nota bene*

- The actual residual has to be equated to 0, not the density. Canonical (translation invariant forms) need to be introduced.
- At a chain length $L = 8$, there are $3^{2L} = 43046721$ matrix entries. It is not feasible, so reduction is needed. Using sparse matrices is a huge performance boost thanks to the nature of the projectors present in the core formula.
- For the variety of the Gröbner basis generated ideal to be finite, one has to flatten the ideal that generates it before letting it solve the system.
- $[H, \mathcal{Q}_5]$ is Hermitian, while $[H, \sum_j \tilde{h}]$ is anti-Hermitian. This needs to be accounted for by putting an i before the $[H, \mathcal{Q}_5]$ in the sum of the two terms: [Re, Im] $\to$ [-Im, Re]. Take $\tilde{h} = A - A^\dagger$ so that $\tilde{h}^\dagger = -\tilde{h}$.
- During the ansatz stage, the $f(a, b)$ is as general as it can be, that is, for each pair a, b, there is a unique h_{j+1} allowed to be turned on. There is no binary-like behaviour.
- Numerics are to be implemented, probably cheap.
- It is still a mystery what the CAS's output will be - will be looked back upon after the specific framework will have been selected.
- Via Gröbner bases, the inclusion of varieties of different ideals can be declared. The gauge symmetries are needed to be sorted out separately, as Gröbner bases are not equipped to handle them.
- There are no interaction patterns considered yet.

- Don't do
$$h = \frac{1}{2}(m + m^\dagger) \tag{3}$$
for hermitization, costs ∼two times as many parameters in the generation procedure.
- Diagonal matrices need to be pruned as early as possible, since they are responsible for large ideal sizes.
-

3 Ansatz module

This module works with an ansatz object so that consistency and restriction checks are run internally, in a self-contained ecosystem. The following functionalities are needed inside the `ansatz.py` module:

- Inputs: hdict (the actual Hamiltonian density), dimension, and whether hermiticity needs to be checked.
- If an a, b combination is not specified, then the constructor adds a zero matrix manually.
- Collecting the symbols then sorting them into a global order.
- Ensuring linearity, then extracting the constant term and the coefficients of the symbols.
- Rejecting non-real symbols.
- Checking whether all the entries are rational.
- Ensuring hermiticity of the ansatz.
- Substituting numerical data in place of the symbols.
- Creating the most general symbolic Hermitian/prespecified block for dimension d.
- Building the actual density out of the ansatz dictionary. Accounts for the following: reflection symmetry of h^{ab}, offdiagonal activity based pattern
- Restricting the ansatz via solving a set of equations as the restrictions. Returns an Ansatz object.

4 Algebra module

Using sparse matrix algebra is mandatory for achieving sensible runtimes. This module is responsible for defining all sorts of operations in this framework. Mathematical transformations of expressions (eg. canonical forms) are also built into the module. The following functionalities are included:

- Treating complex expressions as tuples of two, defining coefficient multiplication (`cmul`) via assigning a new tuple to two preexisting ones.
- Adding real and imaginary parts to existing polynomials stored in dicts via their indices(/keys for sparse).
- Cleaning all zero polynomials from a dictionary.
- Tensor products with the identity both from the left and the right sides L and R times, respectively (also called *padding*).
- Conversion of dict item with tuple-like column and row indicators to square matrices.

- Matrix multiplication.
- Commutator of two matrices.
- Sum/difference of two matrices.
- Commutator density of two translation invariant operators, with densities A, B, where A is placed on a sites and B is placed on b sites. The density is aligned to the left, and lives on N overall sites (for the empty slots, it is padded with $n_{\mathrm{id}} = N - b - a + 1$ identities).
- Computing the canonical form of a translation invariant operator, that is, subtracting a telescopically disappearing $g \otimes \mathbf{1} - \mathbf{1} \otimes g$ from the density.

5 Equations module

Having the ansatz module that creates a dictionary of the Hamiltonian density, the equations determining the integrability properties of the ansatz are obtainable after enforcing the Reshetikhin condition, utilizing the sparse algebraic functionalities of the algebra module. This module can achieve the following:

- The initialization of a system internalizes the ansatz and the dimension (as given parameters), stores and converts the symbols pregiven by the ansatz to CAS-readable format, creates the supplementary $\tilde{h}$ operator's density (which can be one of *diagonal* or *anti-Hermitian*) and creates the polynomial context of the ring generated by the free symbols of both h and $\tilde{h}$.
- Functions that create an arbitrary constant polynomial and a first order polynomial in the k-th variable.
- Construction of the FLINT polynomial entry of a given `const-lin` pair of the ansatz's precomputed linear array entries.
- Conversion of the matrix elements of the ansatz to a FLINT compatible density with cleaning, via splitting (a, b, i, j) to $[(a, i, b), (a, j, b)]$ (this is needed so that it matches the format used by the `algebra` module, which breaks indices down to row and column strings).
- Creating the auxiliary density for $\tilde{h}$. Can add a diagonal or an IRF-type density. In the latter case, $\tilde{h}$ needs to be anti-Hermitian to have a consistent anti-Hermitian commutator $[H, \mathcal{Q}_5]$.
- Generates the equations of the system. It can either use a form for q_5 that is 4-local (can be proven using the diagonal form of the projectors in the ansatz), or 5-local. Computes the auxiliary density's residual ($\mathcal{R}_{\tilde{h}} := [H, [H, \tilde{H}]]$). Converts the density of the residual to a canonical form. Appends every entry's monic polynomial into a list, while checking for redundancies via string-converted keyfinding (this works because the polynomials have definite ordering with respect to the free parameters).
- Performs linear reduction of the polynomial system via monomial linearization.

6 Families module

The ansatz is going to be fed a system of third order polynomial equations, but in the most general case, the number of equations and free parameters is way larger than what can actually be solved. Hence, a systematic reduction in the degrees of freedom is needed - that is why different Hamiltonian families are introduced, each with a unique constraint on the polynomial system. The functionalities of the module:

- `B_RZ2`: Dipolar, real, reflection-invariant Hamiltonian. The extra equations enforced are the ones for $S_{1,\text{flip}} = S_1$, $S_{x,\text{flip}} = S_x$, $S_{z,\text{flip}} = -S_z$. The condition for checking a particular a, b configuration is that the flipped, or the reflected operator's flipped version's canonical key has not yet been appended into the set containing unique keys.
- `Z3`: Start off with a given pair (a, b), then iteratively impose $h^{a+k,b+k}_{i+k,j+k} = h^{ab}_{ij}$. It is obvious that setting a to something other than 0 never introduces new equations: at some k, it reaches $(a + k)$ mod3 $= 0$, which then imposes the same restrictions. This is called a "clock symmetry".
- `Z3`: The same family as above, the only difference being that realness and reflection invariance is imposed on the density.
- `E`: Only letting terms have nonzero offdiagonal components that preserve the magnetization, ie. $m_a + m_b = 0$. For this, each a, b has an assigned magnetization number, therefore enforcing this condition is equivalent to a simple check regarding the indices.

7 Solver module

The 5 prior modules provide the full system of third order polynomial equations, with the full polynomial context, usable by CAS, rational coefficients, linear reduction in the monomials, and the complete symmetrization unique for that specific family. The solver module is essentially a Gröbner basis finder engine, but computing that from the equations, with brute force is unfeasible. Instead, a number of structural partitions (will be spelled out what it means in detail) are imposed upon the system, in the following order, plus the initialization:

- The ansatz generated by the specific family (with the restrictions already encoded into the equations), $\tilde{h}$ and the symmetries regarding *relabeling and reflection*, which are not physical, but rather indicate the degrees of freedom on the choice of labels (here, a physically symmetric family is actually more restricted, eg. if the blocks had reflection symmetry, a non-reflecticve relabeling of the basis states destroys the symmetry built into the system). Then the equations are generated, and by default, undergo monomial linear reduction.
- Finding trivial solutions costs a large amount of time, therefore they need to be filtered out a priori, by hand. These include three subcategories:
  1. Blocks that are **diagonal** (**inactive**) provide $(d-1)d/2$ linear equations that can be used as constraints (saturation) for the *variety*. This way, no block that has $h^{ab}_{i<j} \not\equiv 0$ (non-diagonal/**active**) is accidentally included at a special point where it would reduce to being diagonal. This is useful for finishing off easy components quickly, while also separating the largest case (when the pattern contains every active candidate block). (Also, an inactive block is one which cannot flip a basis state of the 3-site Hilbert space, therefore it is also physically motivated to separate them from state-flipping ones.)
  2. Blocks that have at most **two-local** terms in their densities are not interesting solutions of 3-site models.
  3. Blocks that have trivial $\mathcal{Q}_5 = \equiv 0$ components do not contribute anything to integrability.

  Offdiagonal entries (which are *linear forms*) therefore have an important role in determining trivial points. A set of forms for a given block is linear reduced before being handed over for ideal saturation.

A given ansatz has a number $s \in \{1, ..., d^2\}$ possible active blocks. The disjoint union of solutions sets with different number of active blocks span the whole solution set, therefore they can be checked one by one. One arrangement of active blocks is called a *pattern*, the number of patterns is obtained with the combination $\binom{d^2}{2} - 1$ (-1 for the trivial empty set).

- For a given ansatz, the active blocks are collected (symmetries for a given system determine this set).

- The $\binom{|\text{active}|^2}{2} - 1$ combinations are the candidates for being an active pattern. However, if the system is symmetric to relabelings or reflections, then a family of patterns would correspond to the same solution - these familes are called *orbits*, and only one representative needs to be taken into account in one orbit, per pattern. Also, the patterns need to be checked for consistency: if two forms provide the same equations (therefore spanning the same solution set), then they need to be simultaneously active or inactive.

- Everything above provides constraining linear equations, that will quickly eliminate variables from the Gröbner solver. A *stratum* is the set of solutions for the system of equations constrained by some activity pattern S:

$$V_S := \{x \in V_I : F_{ij}^{ab}(x) \equiv 0 \ \forall i < j \Leftrightarrow k \notin S\}, \tag{4}$$

  where $F_{ij}^{ab}(x)$ is the linear form of matrix entry ij at block (a, b). The disjoint union of these strata provides the complete solution set (the fact that two strata is disjoint can be seen from the fact that no two activity patterns can be the same).

- Before obtaining the equations of a unique stratum, we further partition the system. Exploiting the linear forms' homogeneity, it is obvious that if x is a solution of $\{F_{ij}^{ab} = 0\}_{i<j}$, then also is λx. This freedom can be used to collapse the ray of the solutions to a single value of λ by setting one parameter to a unique number, eg. 1. For this, one of the active blocks will be used, in which the first nonzero form will be set to 1: $F_c(x) = 1$ (the unique index c, encoding the ij matrix index, real or imaginary, corresponds to a **chart**), where if $m < c$, $F_m(x) = 0$. This provides additional linear equations, with the crucial property that the disjoint union of the different c labels meets the solution set at exactly one point for a given first block of a unique activity pattern:

$$U_c := \{x \in V_S : F_m(x) = 0, \text{ if } m < c, F_c(x) = 1\}, \tag{5}$$

$$V_S \cap \bigcup_{c \in \{ij\}_{i<j}} U_c = x_0, \text{ with } x_0 \text{ unique.} \tag{6}$$

  To sum up, a chart consists of the restrictions imposed by the inactive blocks and all the forms preceding c and $F_c(x) = 0$, while the other active blocks will provide the